CS329M · CHECKPOINT · OCTOBER 27, 2026

codemem

Local code search for coding agents, measured against grep

```
$ codemem init --agents claude
$ codemem search \
  "disabled users stay logged in"
```

Wilson Ying · Andre Lin · Andrew Zhang

THE PROBLEM

How coding agents find code

Claude Code and Codex grep. Cursor builds an index. Each says it works better.

GREP AND READ FILES

Claude Code, OpenAI Codex

"Agentic search generally works better."[3]

- Always current: it reads the files
- Many tries on big repos, slow on large files
- Each try costs time, and tokens

Is grep the best way to find code, or.. to sell tokens..? 🤔

SEARCH INDEX

Cursor

"Semantic search is currently necessary to achieve the best results."[1]

- 12.5% higher answer accuracy on average[1]
- Goes stale until re-embedded[2], one reason Claude Code dropped its index[3]

The gap: we found no study that measures how much staleness hurts, or what it costs to keep an index fresh.

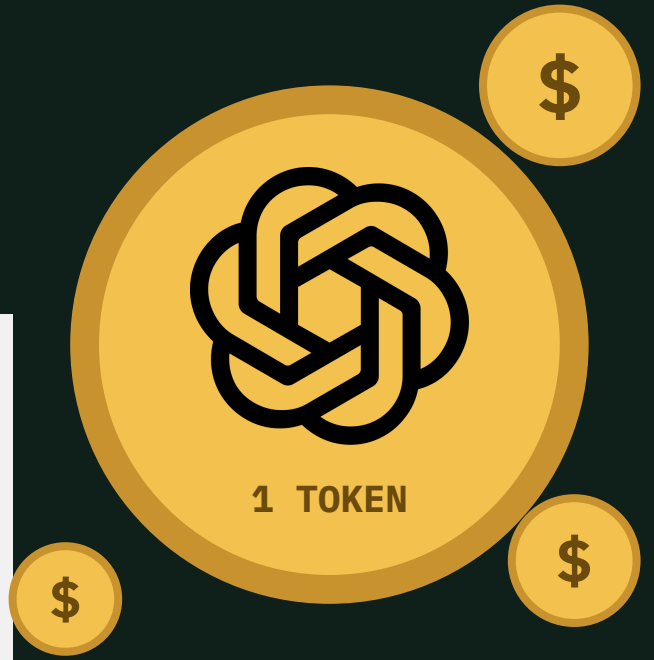
[1]Cursor blog, Nov 6, 2025 [2]Cursor blog, Jan 27, 2026 [3]Boris Cherny (Claude Code), post on X, Jan 31, 2026

More Tries Means More Tokens

And Who Sells The Tokens?

IPO

ANTHROP\C



OUR TOOL

codemem

Ask in plain language. Get a short ranked list of places in the code.

9
languages
Python, JavaScript, TypeScript, Go, Java, C, C++, CUDA, Rust

MCP
for agents
4 tools for Claude Code, Cursor and Codex

CLI
for people
search, index, status and watch, also for scripts

local
by default
Code stays on your machine unless you pick an embedding API

WHAT IT IS FOR

“I want to add login auth, where do I start?”

“Add OCR to my upload pipeline.”

Keyword search fails on these, and grep takes many tries.

codemem works next to grep, which stays the tool for exact strings.

THE CHALLENGE

The catch: an index goes stale

Stale means the index lags behind the code, and agents change code fast.

OPTION 1

Wait until the agent stops editing?

Too slow. The agent edits several files, then searches at once. It gets the old code back and thinks its change is missing.

OPTION 2

Re-embed on every save?

Wasted work. The agent saves the same file again and again, and most new vectors are replaced before anyone searches.

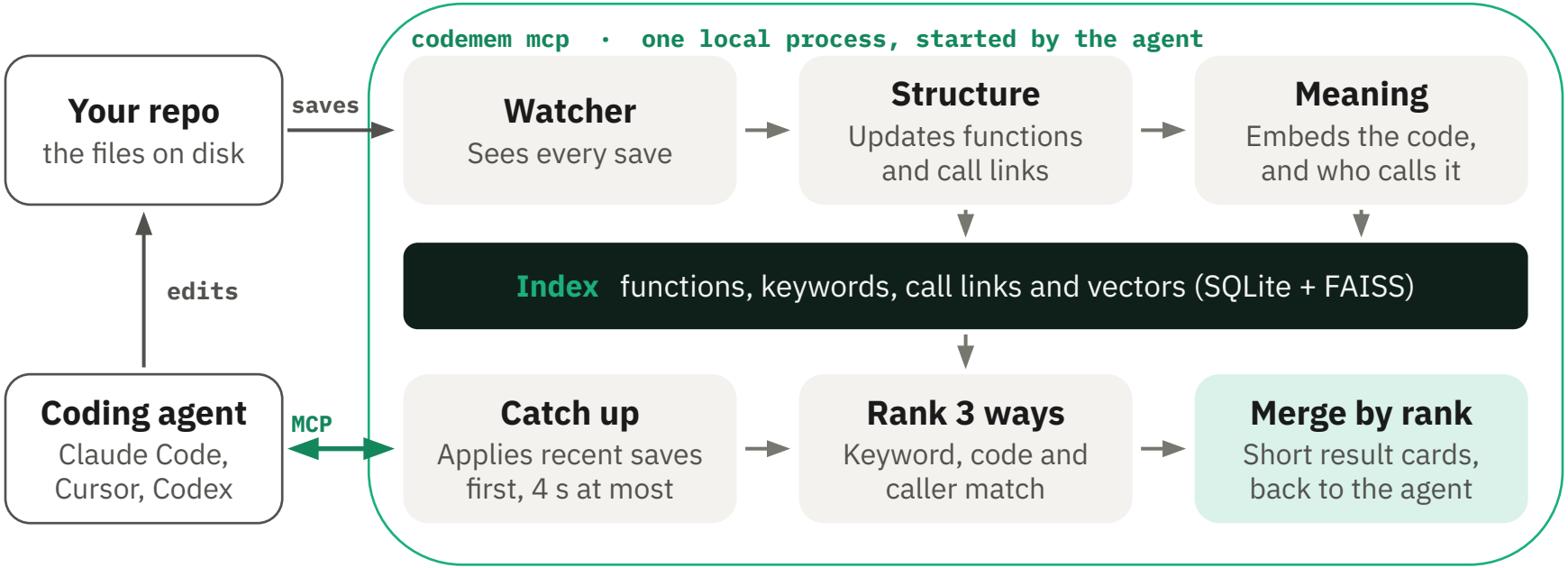
What we need: search results that include the agent's latest edit, with no wasted work and no extra steps for the agent. It works next to grep, not instead of it.

More on that later...

ARCHITECTURE

How codemem works

The agent calls codemem over MCP. codemem watches the repo to stay current.



Connect once: `codemem init --agents claude` writes .mcp.json. Claude Code then starts codemem in every session.

FRESHNESS

Keeping the index fresh

Every search first catches up on the agent's own edits, for at most 4 s.

1

Save

The watcher picks it up

2

Structure

Keywords, call links and lines current in milliseconds

3

Meaning

New vectors for edited code: 0.5 to 4 s for a typical fix

4

Search

Waits 4 s at most, then names any file still pending

`Or see the interactive demo for how it works!`

https://wyy1048576.github.io/codemem/explorer/#/big/0

Measured in the eval runs, Mac M5 GPU. A change of over 1,000 chunks, such as a branch switch, skips the wait and reports what is pending.

EVALUATION

Does codemem beat grep?

218 fresh tasks from 9 big repos, in **Python**, **C++** and **TypeScript**
And more to come!

EVALUATION

What we measure

Given only the issue text, find the code that the real fix changed.

NAIVE BASELINE

grep-fn

Searches like an agent: grep, then open the function around each match. Ranked by match count.

STRONG BASELINE

grep-fn-plus

Our tuned grep: rare words weigh more, noise words dropped, tests last. Reads only the task's language.

OURS

codemem

Keywords, code meaning and call links. Default settings, not tuned. Reads every language.

Accuracy (MRR)

Is the right function near the top of the list?

Our guess: similar to grep/slightly better

Tokens

How much an agent reads before the right function

Our guess: far fewer

Latency

Time per search, and time to build the index

Our guess: faster

Dev split: 218 tasks in 9 repos (Python 100, C++ 68, TypeScript 50). Grading needs no LLM, so it has no API cost. grep-fn-plus rules were set on Python tasks. An agent could get them once from CLAUDE.md; we plan to test this.

EVALUATION

How we made the tasks

Every task is a real fix, merged after the model's training cutoff.

STEP 1

33,568

merged PRs

9 big repos, July to October 2026

STEP 2

5,334

one linked issue

written before the fix, with no leak of it

STEP 3

3,788

patch rebuilt

on the code just before the fix; small fixes only

STEP 4

3 levels

gold labels

files, classes and functions the fix changed

STEP 5

218

dev tasks run

25 per repo, from the 20% dev split

CHECKED

15 of 15
tasks match GitHub, checked by hand

Same results
from an updated index and a rebuilt one

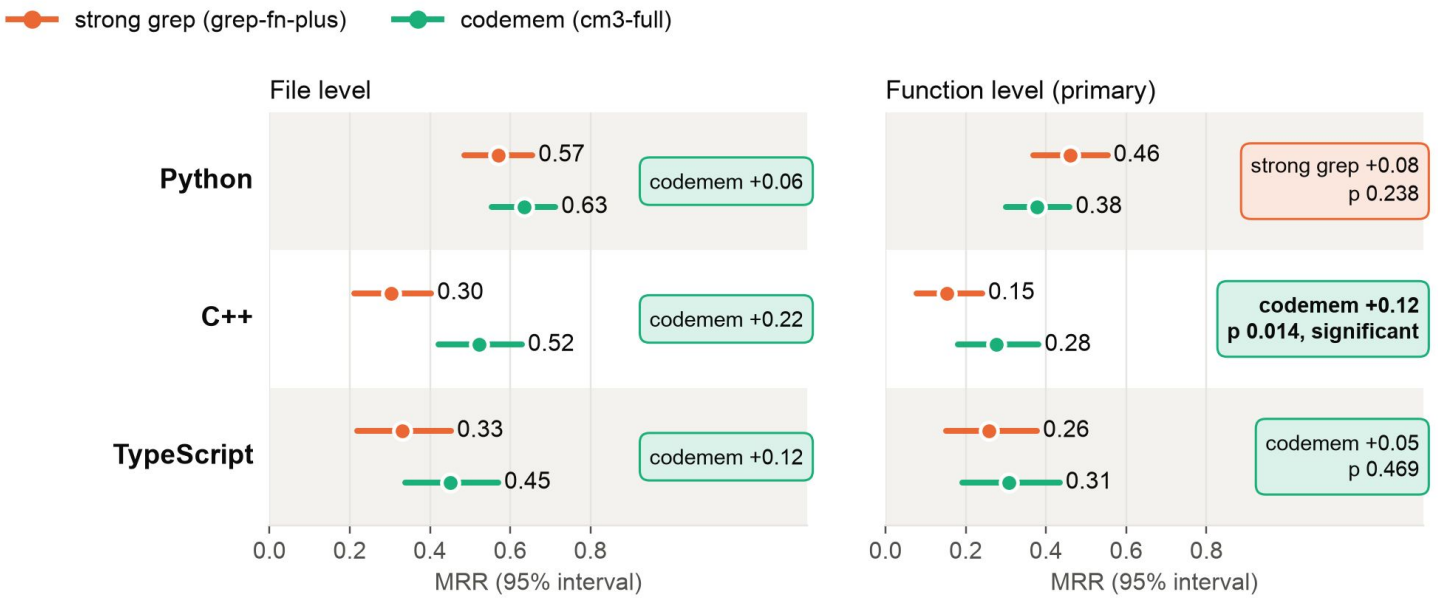
Byte for byte
every report rebuilds from the result files

Step by step: azhang03.github.io/eval-viz

RESULTS

Results: accuracy

codemem ranks the right file higher in all 3 languages. Functions: it depends.

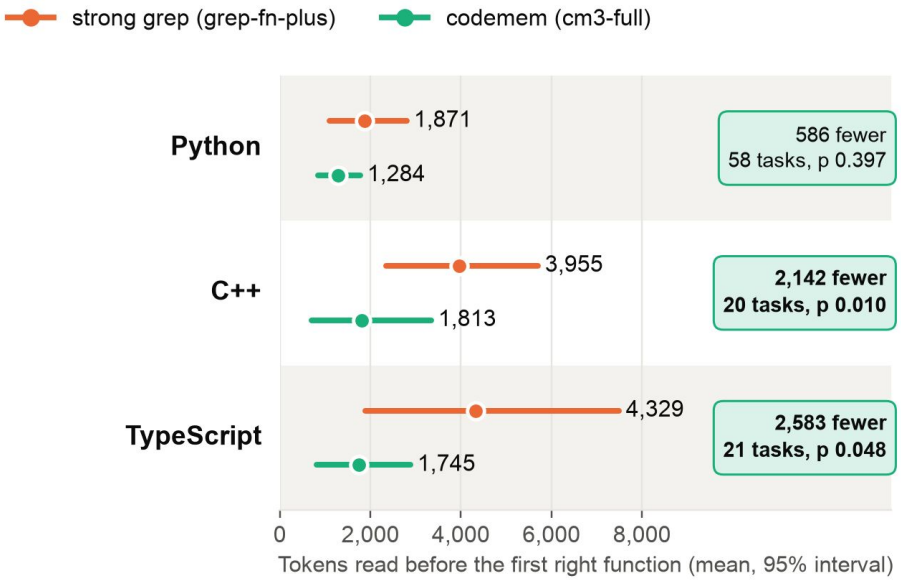


MRR: 1 / rank of the first right result. p: Holm over the 2 primary tests per language. Dev split.

RESULTS

Results: tokens and latency

codemem reads fewer tokens to the answer and searches 2.5 to 4 times faster.



MEDIAN TIME PER SEARCH

	grep-fn-plus	codemem
Python	2.5 s	**1.0 s**
C++	4.8 s	**1.2 s**
TypeScript	2.9 s	**0.7 s**

THE COST

First index: 12 min to 2 h per repo
Each later edit: about 0.5 to 4 s

Tokens: mean on the tasks both systems found. p: Holm over 2 tests per language. Dev split, Mac M5 GPU.

TRADEOFFS

The price: disk space

Each indexed function, class or block takes 10 to 14 KB: 0.25 to 2.35 GB per repo.

(Big) REPO	LANGUAGE	FILES	LINES	INDEX ON DISK	FIRST BUILD
Godot	C++	8.1k	4.6M	**2.35 GB**	121 min
vscode	TypeScript	12.0k	3.5M	**2.17 GB**	95 min
home-assistant	Python	17.9k	3.4M	**≈1.6 GB**	66 min
transformers	Python	4.6k	1.7M	**≈1.1 GB**	45 min
duckdb	C++	5.0k	1.6M	**1.01 GB**	43 min
airflow	Python	7.6k	1.5M	**≈1.0 GB**	40 min
pandas	Python	1.6k	0.7M	**0.38 GB**	16 min
llama.cpp	C++	1.2k	0.7M	**≈0.32 GB**	19 min
playwright	TypeScript	1.4k	0.5M	**0.25 GB**	12 min

Projected size on disk after codemem closes: the measured sizes minus the keyword index's copy of the code, which codemem no longer keeps (12% of an index on vLLM). ≈: also estimated from the size while open (79%, the average of the 5 we re-measured). Files and lines in the main language. First build with an empty vector cache, Mac M5 GPU; with cached vectors, 7 s to 146 s.

TRADEOFFS

Don’t worry: both costs can shrink

These repos are huge, and several of codemem’s parts are still naive.

DISK AND MEMORY

1.5 to 2.5× smaller

Vectors, 71%: stored twice today. Keep one copy, in fewer bytes. Possibly quantized.

Godot: 2.35 GB → ~1 GB
****Depends** on the methods and on the embeddings distribution.

FIRST BUILD

Paid once, and it can be faster

Once per machine: after it, each edit takes 0.5 to 4 s.

Embedding API: a bigger model on fast GPUs, about $1 to $2 for all of Godot.

Shared vectors: with a teammate’s index, Cursor’s slowest first searches fell from 4 h to 21 s[1]

Shares measured on vLLM’s index (102k chunks, 1.16 GB). Godot is about 80M tokens to embed; Qwen3-Embedding-8B lists at $0.01 to $0.02 per million.

[1] Cursor blog, Jan 27, 2026

NEXT

Stretch goals

Two tracks: a wider, more realistic evaluation, and a leaner, faster codemem.

EVALUATION

Full suite, more languages and sizes

Medium repos (a bit bigger than normal), and **big ones** like Godot (~4.6M lines) and home-assistant (~3.4M)

Real agents, end to end

Today we score retrieval only. Next: an **agent with codemem**, **grep** against **grep** alone, considering `index.md` effects too. Count **a) exact tokens** and **b) time from prompt to commit**

Close the model gap?

Top models cost too much to run 24/7. Could a cheaper model with codemem match them? Grep floods weaker models with context, which feeds **geometrically constrained stochasticity**

OPTIMIZATION

Memory

Vectors stored once, and quantized: 1.5-2.5x smaller

Retrieval

Several vectors per function, not one / multi-vector or token-level embedding

Latency

Faster first builds and searches

A NEW VISION

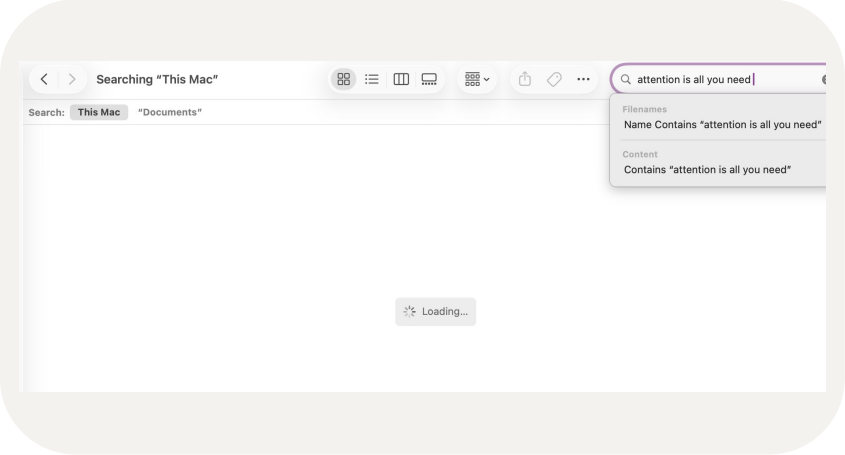
Search By What You Mean.
Not What You Type!

Scaling beyond code

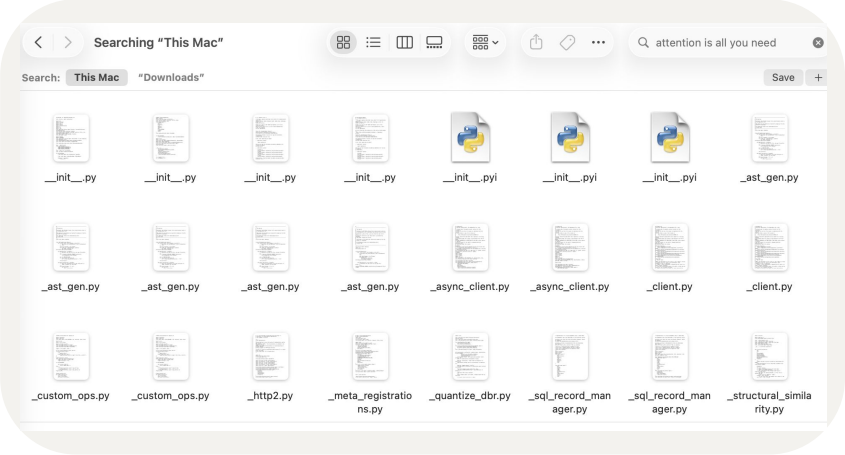
We built codemem for codebases, but anything can be indexed and searched.

Information Exodus Era: bigger codebases, and more files and notes for agents to search.

What if codemem indexed your whole computer, for smarter, faster search?



...takes awhile to load..



Finder: "attention is all you need" finds **random .py files**...

THANK YOU

Questions?

```
$ codemem search "questions?"
```

Eval, step by step: azhang03.github.io/eval-viz

Wilson Ying
MCP connectors for any agent, installation and usability

Andre Lin
Backend infrastructure, system design and research

Andrew Zhang
Eval suite and benchmarking